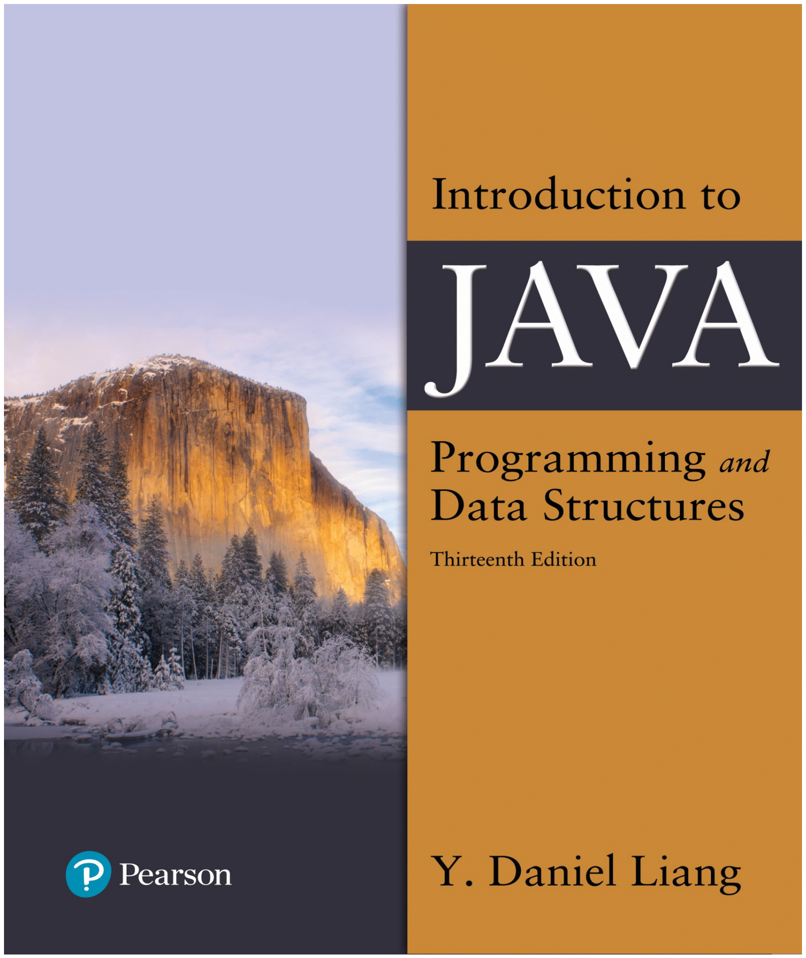
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 10

Thinking in Objects

Review: Objects and Classes
Money Class

- A Money object can be represented by the following UML:

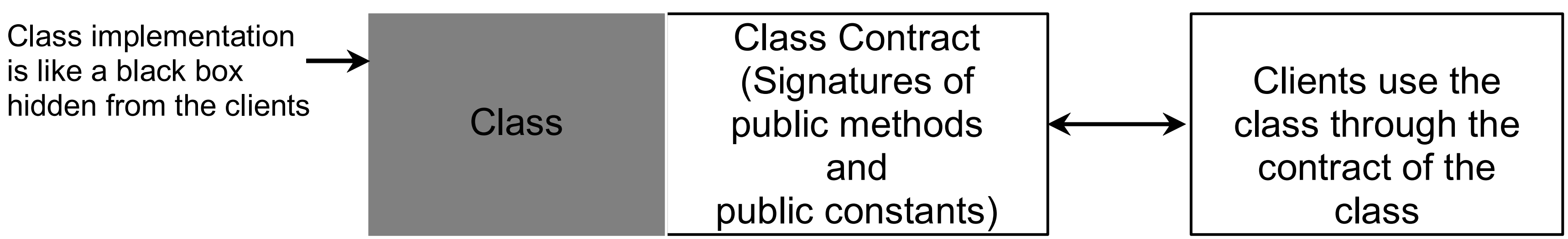
Money
– dollars : int
– cents : int (Must be 0 to 99)
+ Money() // default value 0.0
+ Money(dollar : int, cent : int)
+ getDollars() : int
+ setDollars(dollar : int) : void
+ getCents() : int
+ SetCents(cent : int) : void
+ add(m : Money) : Money
+ subtract(m : Money) : Money // return null if m is greater than this object
+ equals(m : Money) : Boolean
+ compareTo(m : Money) : int
+ printList(list : Money[] , n : int) : void // print n values per line separated by 1 blank space
+ selectionSort(list : Money[]) : void // sort Money array using selection sort
+ max(list : Money[]) : Money // return the largest Money object
+ toString() : String // format is: $#.##

Motivations

You see the advantages of object-oriented programming from the preceding chapter. This chapter will demonstrate how to solve problems using the object-oriented paradigm.

Class Abstraction and Encapsulation

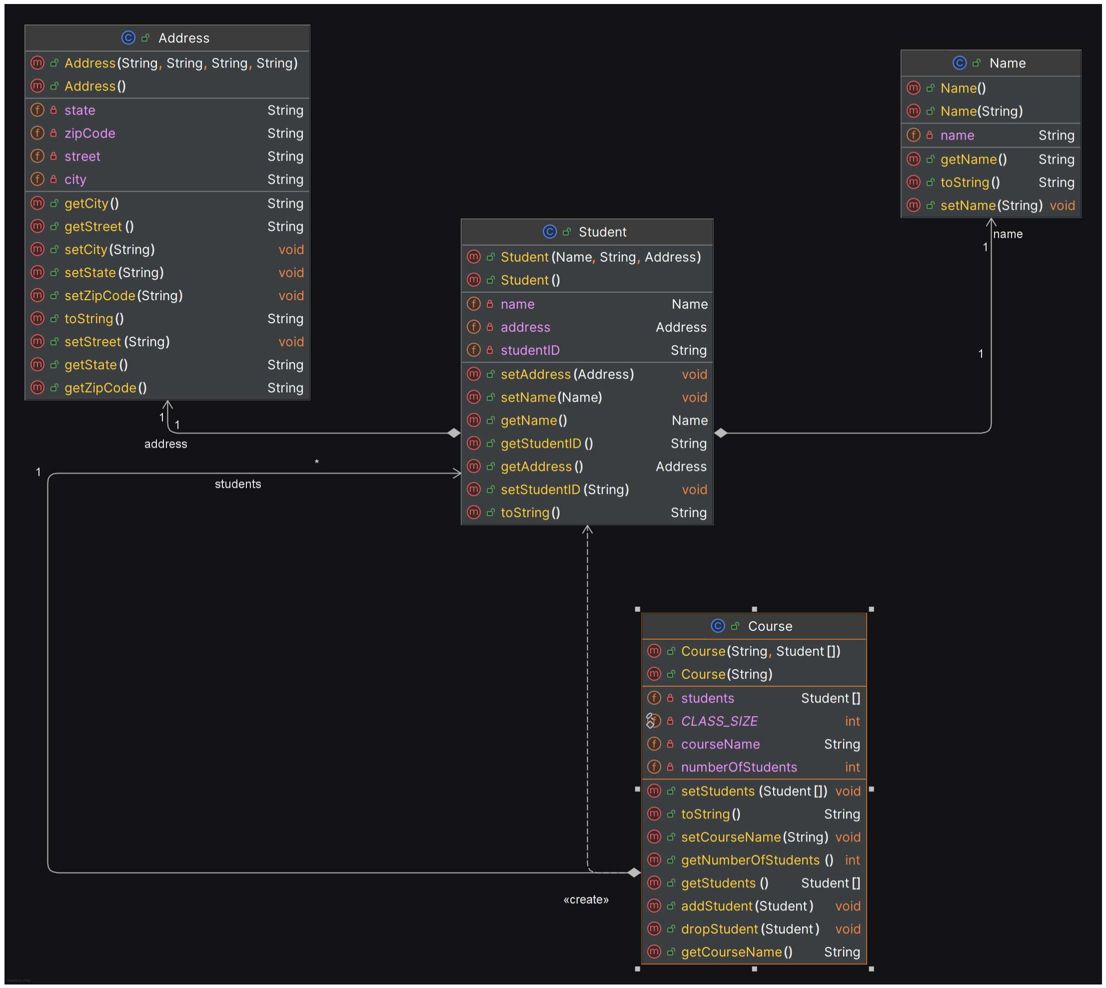
Class abstraction means to separate class implementation from the use of the class. The creator of the class provides a description of the class and let the user know how the class can be used. The user of the class does not need to know how the class is implemented. The detail of implementation is encapsulated and hidden from the user.



Object-Oriented Thinking

Chapters 1-8 introduced fundamental programming techniques for problem solving using loops, methods, and arrays. The studies of these techniques lay a solid foundation for object-oriented programming. Classes provide more flexibility and modularity for building reusable software. This section improves the solution for a problem introduced in Chapter 3 using the object-oriented approach. From the improvements, you will gain the insight on the differences between the procedural programming and object-oriented programming and see the benefits of developing reusable code using objects and classes.

Course UML



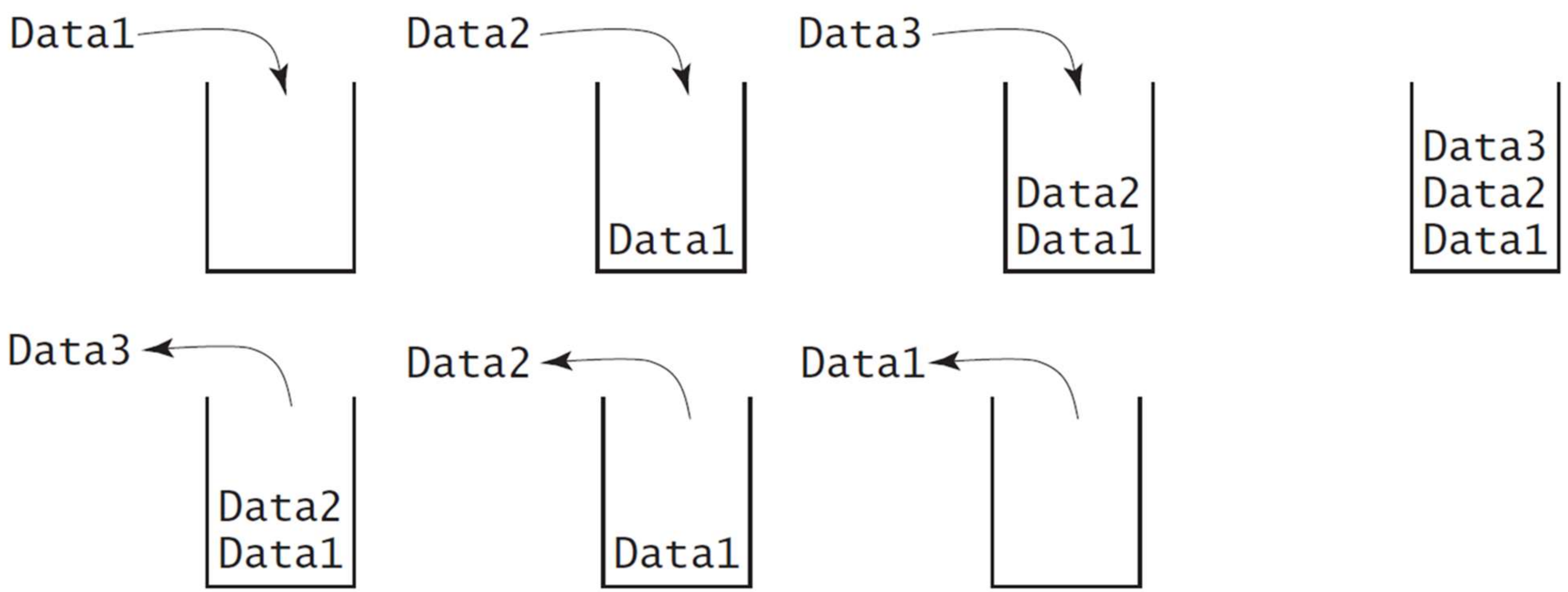
Address
Address(String, String, String, String)
Address()
state String
zipCode String
street String
city String
getCity() String
getStreet() String
setCity(String) void
setState(String) void
setZipCode(String) void
toString() String
setStreet(String) void
getState() String
getZipCode() String
Name
Name()
Name(String)
name String
getName() String
toString() String
setName(String) void
Student
Student(Name, String, Address)
Student()
name Name
address Address
studentID String
setAddress(Address) void
setName(Name) void
getName() Name
getStudentID() String
getAddress() Address
setStudentID(String) void
toString() String
Course
Course(String, Student[])
Course(String)
students Student[]
CLASS_SIZE int
courseName String
numberOfStudents int
setStudents(Student[]) void
toString() String
setCourseName(String) void
getNumberOfStudents() int
getStudents() Student[]
addStudent(Student) void
dropStudent(Student) void
getCourseName() String
address
students
name
«create»

Example: The `StackOfIntegers` Class

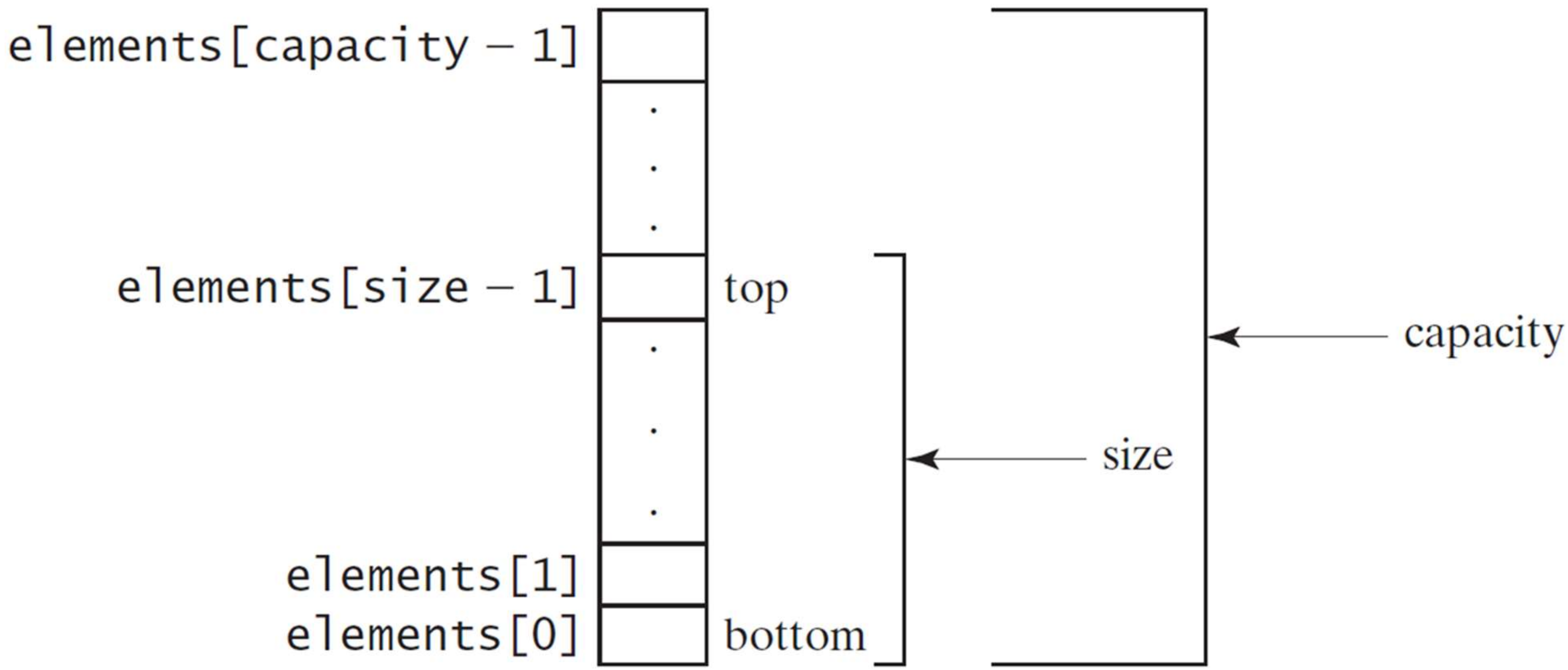
StackOfIntegers	
-elements: int[]	An array to store integers in the stack.
-size: int	The number of integers in the stack.
+StackOfIntegers()	Constructs an empty stack with a default capacity of 16.
+StackOfIntegers(capacity: int)	Constructs an empty stack with a specified capacity.
+empty(): boolean	Returns true if the stack is empty.
+peek(): int	Returns the integer at the top of the stack without removing it from the stack.
+push(value: int): int	Stores an integer into the top of the stack.
+pop(): int	Removes the integer at the top of the stack and returns it.
+getSize(): int	Returns the number of elements in the stack.

TestStackOfIntegers

Designing the StackOfIntegers Class



Implementing `StackOfIntegers` Class



StackOfIntegers

Wrapper Classes

- Boolean
- Character
- Short
- Byte
- Integer
- Long
- Float
- Double

Note: (1) The wrapper classes do not have no-arg constructors. (2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

The `Integer` and `Double` Classes

java.lang.Integer	java.lang.Double
-value: int	-value: double
+MAX_VALUE: int	+MAX_VALUE: double
+MIN_VALUE: int	+MIN_VALUE: double
+Integer(value: int)	+Double(value: double)
+Integer(s: String)	+Double(s: String)
+byteValue(): byte	+byteValue(): byte
+shortValue(): short	+shortValue(): short
+intValue(): int	+intValue(): int
+longVlaue(): long	+longVlaue(): long
+floatValue(): float	+floatValue(): float
+doubleValue():double	+doubleValue():double
+compareTo(o: Integer): int	+compareTo(o: Double): int
+toString(): String	+toString(): String
+valueOf(s: String): Integer	+valueOf(s: String): Double
+valueOf(s: String, radix: int): Integer	+valueOf(s: String, radix: int): Double
+parseInt(s: String): int	+parseDouble(s: String): double
+parseInt(s: String, radix: int): int	+parseDouble(s: String, radix: int): double

The Static valueOf Methods

The numeric wrapper classes have a useful class method, valueOf(String s). This method creates a new object initialized to the value represented by the specified string. For example:

```
Double doubleObject = Double.valueOf("12.4");

Integer integerObject = Integer.valueOf("12");
```

Automatic Conversion Between Primitive Types and Wrapper Class Types

JDK 1.5 allows primitive type and wrapper classes to be converted automatically. For example, the following statement in (a) can be simplified as in (b):

```
Integer[] intArray = {new Integer(2),
  new Integer(4), new Integer(3)};
```

(a)

Equivalent

```
Integer[] intArray = {2, 4, 3};
```

(b)

New JDK 1.5 boxing

```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Unboxing

The `String` Class

- Constructing a String:

```
String message = "Welcome to Java“;
String message = new String("Welcome to Java“);
String s = new String();
```

- Obtaining String length and Retrieving Individual Characters in a string
- String Concatenation (concat)
- Substrings (substring(index), substring(start, end))
- Comparisons (equals, compareTo)
- String Conversions
- Finding a Character or a Substring in a String
- Conversions between Strings and Arrays
- Converting Characters and Numeric Values to Strings

Constructing Strings

String newString = new String(stringLiteral);

String message = new String("Welcome to Java");

Since strings are used frequently, Java provides a shorthand initializer for creating a string:

String message = "Welcome to Java";

Strings Are Immutable

A String object is immutable; its contents cannot be changed. Does the following code change the contents of the string?

```
String s = "Java";
s = "HTML";
```

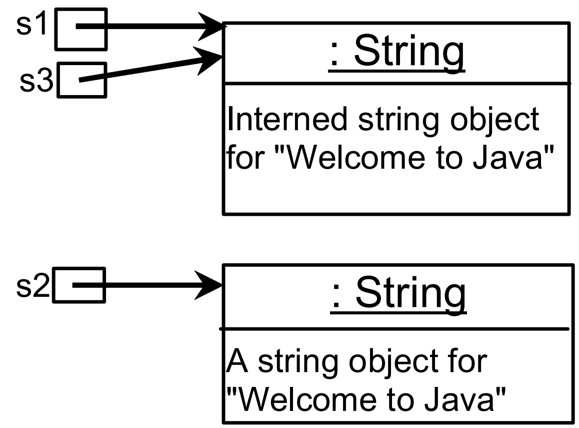
Interned Strings

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

s1 == s is false

s1 == s3 is true

A new object is created if you use the new operator.

If you use the string initializer, no new object is created if the interned object is already created.

Replacing and Splitting Strings (1 of 2)

java.lang.String	
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Splitting a String

```
String[] tokens =
"Java#HTML#Perl".split("#");

for (int i = 0; i < tokens.length; i++)

  System.out.print(tokens[i] + " ");
```

displays

Java HTML Perl

Matching, Replacing and Splitting by Patterns (1 of 3)

You can match, replace, or split a string by specifying a pattern. This is an extremely useful and powerful feature, commonly known as **regular expression**. Regular expression is complex to beginning students. For this reason, two simple patterns are used in this section. Please refer to Supplement III.F, “Regular Expressions,” for further studies.

```
"Java".matches("Java");
"Java".equals("Java");
"Java is fun".matches("Java.*");
"Java is cool".matches("Java.*");
```

Matching, Replacing and Splitting by Patterns (2 of 3)

The replaceAll, replaceFirst, and split methods can be used with a regular expression. For example, the following statement returns a new string that replaces $, +, or # in "a+b$#c" by the string NNN.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");

System.out.println(s);
```

Here the regular expression [$+#] specifies a pattern that matches $, +, or #. So, the output is aNNNbNNNNNNc.

Matching, Replacing and Splitting by Patterns (3 of 3)

The following statement splits the string into an array of strings delimited by some punctuation marks.

```
String[] tokens = "Java,C?C#,C++".split("[.,:;?]");

for (int i = 0; i < tokens.length; i++)

System.out.println(tokens[i]);
```

StringBuilder

The `StringBuilder` class is an alternative to the `String` class. In general, a StringBuilder can be used wherever a string is used. StringBuilder is more flexible than String. You can add, insert, or append new contents into a string buffer, whereas the value of a String object is fixed once the string is created.

`StringBuilder` Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: *aPrimitiveType*): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Examples

```
StringBuilder s = new StringBuilder();
s.append("Welcome to Java");
System.out.println(s.toString());
s.insert(11,"C++ and ");
System.out.println(s.toString());
s.deleteCharAt(1);
System.out.println(s.toString());
s.insert(1,"e");
System.out.println(s.toString());
s.replace(11,14,"CPP");
System.out.println(s.toString());
System.out.println(s.reverse());
```

The toString, capacity, length, setLength, and charAt Methods

java.lang.StringBuilder	
+toString(): String	Returns a string object from the string builder.
+capacity(): int	Returns the capacity of this string builder.
+charAt(index: int): char	Returns the character at the specified index.
+length(): int	Returns the number of characters in this builder.
+setLength(newLength: int): void	Sets a new length in this builder.
+substring(startIndex: int): String	Returns a substring starting at startIndex.
+substring(startIndex: int, endIndex: int): String	Returns a substring from startIndex to endIndex-1.
+trimToSize(): void	Reduces the storage size used for the string builder.

Problem: Checking Palindromes Ignoring Non-alphanumeric Characters

This example gives a program that counts the number of occurrence of each letter in a string. Assume the letters are not case-sensitive.

PalindromeIgnoreNonAlphanumeric

Pearson

Regular Expressions

A **regular expression** (abbreviated **regex**) is a string that describes a pattern for matching a set of strings. Regular expression is a powerful tool for string manipulations. You can use regular expressions for matching, replacing, and splitting strings.

Matching Strings

```
"Java".matches("Java");

"Java".equals("Java");

"Java is fun".matches("Java.*")

"Java is cool".matches("Java.*")

"Java is powerful".matches("Java.*")
```

Regular Expression Syntax

Regular Expression	*Matches*	*Example*
x	a specified character x	Java matches Java
.	any single character	Java matches J..a
(ab|cd)	ab or cd	ten matches t(en|im)
[abc]	a, b, or c	Java matches Ja[uvwx]a
[^abc]	any character except a, b, or c	Java matches Ja[^ars]a
[a-z]	a through z	Java matches [A-M]av[a-d]
[^a-z]	any character except a through z	Java matches Jav[^b-d]
[a-e[m-p]]	a through e or m through p	Java matches [A-G[I-M]]av[a-d]
[a-e&&[c-p]]	intersection of a-e with c-p	Java matches [A-P&&[I-M]]av[a-d]
\d	a digit, same as [0-9]	Java2 matches "Java[\\d]"
\D	a non-digit	$Java matches "[\\D][\\D]ava"
\w	a word character	Java1 matches "[\\w]ava[\\w]"
\W	a non-word character	$Java matches "[\\W][\\w]ava"
\s	a whitespace character	"Java 2" matches "Java\\s2"
\S	a non-whitespace char	Java matches "[\\S]ava"
*p**	zero or more occurrences of pattern *p*	aaaabb matches "a*bb" ababab matches "(ab)*"
p+	one or more occurrences of pattern *p*	a matches "a+b*" able matches "(ab)+.*"
p?	zero or one occurrence of pattern *p*	Java matches "J?Java" Java matches "J?ava"
p{n}	exactly n occurrences of pattern *p*	Java matches "Ja{1}.*" Java does not match ".{2}"
p{n,}	at least n occurrences of pattern *p*	aaaa matches "a{1,}" a does not match "a{2,}"
p{n,m}	between n and m occurrences (inclusive)	aaaa matches "a{1,9}" abb does not match "a{2,9}bb"

Pearson

Replacing and Splitting Strings (2 of 2)

java.lang.String	
+matches(regex: String): boolean	Returns true if this string matches the pattern.
+replaceAll(regex: String, replacement: String): String	Returns a new string that replaces all matching substrings with the replacement.
+replaceFirst(regex: String, replacement: String): String	Returns a new string that replaces the first matching substring with the replacement.
+split(regex: String): String[]	Returns an array of strings consisting of the substrings split by the matches.

Examples (4 of 4)

// replace v followed by any word char. With “wi”

String s = "Java Java Java".replaceAll("v\\w", "wi") ;

S → “Jawi Jawi Jawi”

String s = "Java Java Java".replaceFirst("v\\w", "wi") ;

S → “Jawi Java Java”

String[] s = "Java1HTML2Perl".split("\\d");

https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html